

目录

基本模版	1
Markov 不等式	1
Chebyshev 不等式	1
简介	2
定义	2
引理	2
本篇记号声明	2
无向图情况	2
有向图情况	3
定理叙述	3
BEST 定理	4
一般图最大权匹配	4
一般图最大独立集	9
全图割	10

基本模版

```
#define gep(k,from,Head,Er) for(int k=Head[from];k!=-1;k=Er[k].nxt)
#define rep(i,a,b) for(int i=a,rep##i=b;i<=rep##i;i++)
#define per(i,a,b) for(int i=a,rep##i=b;i>=rep##i;i--)
#define sz(x) (static_cast<int>(x.size()))
#define all(x) x.begin(),x.end()
template<typename T>void Clear(T&x){T y;x.swap(y);}
```

1. 添加数组: `*(type(*)[num])array`
2. `(long long(*)[20])A._M_impl._M_start`

```
g++ Sol.cpp -Wall -std=c++23 -g -Wl,-stack_size -Wl,0x20000000 -o Sol -DLOCAL
```

Markov 不等式

设 X 是一个取值非负的随机变量, 则对任意正实数 a 有

$$P\{X \geq a\} \leq \frac{EX}{a}$$

Chebyshev 不等式

设 X 是一随机变量, 则对任意的 $a > 0$ 都有

$$P\{|X - EX| \geq a\} \leq \frac{DX}{a^2}$$

特别地, 当 a 取 $k\sigma$ 时有

$$P\{|X - EX| \geq k\sigma\} \leq \frac{1}{k^2}$$

其中 σ 是 X 的标准差。

简介

Lindström-Gessel-Viennot lemma, 即 LGV 引理, 可以用来处理有向无环图上不相交路径计数等问题。

前置知识: 图论相关概念 中的基础部分、矩阵、高斯消元求行列式。

LGV 引理仅适用于 有向无环图。

定义

$\omega(P)$ 表示 P 这条路径上所有边的边权之积。(路径计数时, 可以将边权都设为 1) (事实上, 边权可以为生成函数)

$e(u, v)$ 表示 u 到 v 的 每一条 路径 P 的 $\omega(P)$ 之和, 即 $e(u, v) = \sum_{P: u \rightarrow v} \omega(P)$ 。

起点集合 A , 是有向无环图点集的一个子集, 大小为 n 。

终点集合 B , 也是有向无环图点集的一个子集, 大小也为 n 。

一组 $A \rightarrow B$ 的不相交路径 S : S_i 是一条从 A_i 到 $B_{\sigma(S)_i}$ 的路径 ($\sigma(S)$ 是一个排列), 对于任何 $i \neq j$, S_i 和 S_j 没有公共顶点。

$t(\sigma)$ 表示排列 σ 的逆序对个数。

引理

$$M = \begin{bmatrix} e(A_1, B_1) & e(A_1, B_2) & \cdots & e(A_1, B_n) \\ e(A_2, B_1) & e(A_2, B_2) & \cdots & e(A_2, B_n) \\ \vdots & \vdots & \ddots & \vdots \\ e(A_n, B_1) & e(A_n, B_2) & \cdots & e(A_n, B_n) \end{bmatrix}$$

$$\det(M) = \sum_{S: A \rightarrow B} (-1)^{t(\sigma(S))} \prod_{i=1}^n \omega(S_i)$$

其中 $\sum_{S: A \rightarrow B}$ 表示满足上文要求的 $A \rightarrow B$ 的每一组不相交路径 S 。

矩阵树定理解决了一张图的生成树个数计数问题。

本篇记号声明

本篇中的图, 无论无向还是有向, 都允许重边, 但是默认没有自环。

??? note “有自环的情形” 自环并不影响生成树的个数, 也不影响下文中 Laplace 矩阵的计算, 故而矩阵树定理对有自环的情形依然成立。计算时不必删去自环。如果删去自环, 会影响根据 BEST 定理应用矩阵树定理统计有向图的欧拉回路个数。

无向图情况

设 G 是一个有 n 个顶点的无向图。定义度数矩阵 $D(G)$ 为

$$D_{ii}(G) = \deg(i), \quad D_{ij} = 0, \quad i \neq j.$$

设 $\#e(i, j)$ 为点 i 与点 j 相连的边数, 并定义邻接矩阵 A 为

$$A_{ij}(G) = A_{ji}(G) = \#e(i, j), \quad i \neq j.$$

定义 Laplace 矩阵（亦称 Kirchhoff 矩阵） L 为

$$L(G) = D(G) - A(G).$$

记图 G 的所有生成树个数为 $t(G)$ 。

有向图情况

设 G 是一个有 n 个顶点的有向图。定义出度矩阵 $D^{out}(G)$ 为

$$D_{ii}^{out}(G) = \deg^{out}(i), \quad D_{ij}^{out} = 0, \quad i \neq j.$$

类似地定义入度矩阵 $D^{in}(G)$ 。

设 $\#e(i, j)$ 为点 i 指向点 j 的有向边数，并定义邻接矩阵 A 为

$$A_{ij}(G) = \#e(i, j), \quad i \neq j.$$

定义出度 Laplace 矩阵 L^{out} 为

$$L^{out}(G) = D^{out}(G) - A(G).$$

定义入度 Laplace 矩阵 L^{in} 为

$$L^{in}(G) = D^{in}(G) - A(G).$$

记图 G 的以 k 为根的所有根向树形图个数为 $t^{\text{root}}(G, k)$ 。所谓根向树形图，是说这张图的基图是一棵树，所有的边全部指向父亲。

记图 G 的以 k 为根的所有叶向树形图个数为 $t^{\text{leaf}}(G, k)$ 。所谓叶向树形图，是说这张图的基图是一棵树，所有的边全部指向儿子。

定理叙述

矩阵树定理具有多种形式。

定义 $[n] = \{1, 2, \dots, n\}$ ，矩阵 A 的子矩阵 $A_{S,T}$ 为选取 $A_{i,j}$ ($i \in S, j \in T$) 的元素得到的子矩阵。

“定理 1（矩阵树定理，无向图，行列式形式）”

对于无向图 G 和任意的 k ，都有

$$t(G) = \det L(G)_{[n] \setminus \{k\}, [n] \setminus \{k\}}.$$

也就是说，无向图的 Laplace 矩阵所有 $n-1$ 阶主子式都相等，且都等于图的生成树的个数。

“推论 1（矩阵树定理，无向图，特征值形式）”

设 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{n-1} \geq \lambda_n = 0$ 为 $L(G)$ 的 n 个特征值，那么有

$$t(G) = \frac{1}{n} \lambda_1 \lambda_2 \cdots \lambda_{n-1}.$$

“定理 2（矩阵树定理，有向图根向树，行列式形式）”

对于有向图 G 和任意的 k ，都有

$$t^{\text{root}}(G, k) = \det L^{\text{out}}(G)_{[n] \setminus \{k\}, [n] \setminus \{k\}}.$$

也就是说，有向图的出度 Laplace 矩阵删去第 k 行第 k 列得到的主子式等于以 k 为根的根向树形图的个数。

因此如果要统计一张图所有的根向树形图，只要枚举所有的根 k 并对 $t^{\text{root}}(G, k)$ 求和即可。

“定理 3（矩阵树定理，有向图叶向树，行列式形式）”

对于有向图 G 和任意的 k ，都有

$$t^{\text{leaf}}(G, k) = \det L^{\text{in}}(G)_{[n] \setminus \{k\}, [n] \setminus \{k\}}.$$

也就是说，有向图的入度 Laplace 矩阵删去第 k 行第 k 列得到的主子式等于以 k 为根的叶向树形图的个数。

因此如果要统计一张图所有的叶向树形图，只要枚举所有的根 k 并对 $t^{\text{leaf}}(G, k)$ 求和即可。

根向树形图也被称为内向树形图，但因为计算内向树形图用的是出度，为了不引起 in 和 out 的混淆，所以采用了根向这一说法。

BEST 定理

前置知识：欧拉图

这一定理将有向欧拉图中欧拉回路的数目和该图的根向树形图的数目联系起来，从而解决了有向图中的欧拉回路的计数问题。注意，任意无向图中的欧拉回路的计数问题是 NP 完全的。

在实现该算法时，应当首先判定给定图是否是欧拉图，移除所有零度顶点，然后建图计算根向树形图的个数，并由 BEST 定理得到欧拉回路的计数。注意，如果所求欧拉回路个数要求以给定点作为起点，需要将答案再乘上该点出度，相当于枚举回路中首条边。

在证明 BEST 定理之前，需要知道如下结论。

一个有向图具有欧拉回路，当且仅当非零度顶点是强连通的，且所有顶点的出度和入度相等。

对于欧拉图，因为出度和入度相等，可以将它们略去上标，记作 $\deg(v)$ 。BEST 定理可以叙述如下。

设 G 是有向欧拉图， k 为任意顶点，那么 G 的不同欧拉回路总数 $\text{ec}(G)$ 是

$$\text{ec}(G) = t^{\text{root}}(G, k) \prod_{v \in V} (\deg(v) - 1)!.$$

这也说明，对欧拉图 G 的任意两个节点 k, k' ，都有 $t^{\text{root}}(G, k) = t^{\text{root}}(G, k')$ 。

一般图最大权匹配

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll Inf = (ll)1e18;
const int MaxV = 410; // 可按需要调整 (>= 实际顶点数)
```

```

struct Edges {
    int from, to;
    int data;
    Edges(int _from = 0, int _to = 0, int _data = 0) : from(_from), to(_to), data(_data) {}
};
int sizev; // 原图顶点数（不含“花”的扩展节点）
int tot;   // 当前编号的最大节点（包括扩展出来的花节点）
vector<vector<Edges>> Eij; // Eij[i][j] 保存 i->j 的边信息（用矩阵语义便于花合并时更新）
vector<vector<int>> Root; // Root[flower][j] = 哪个原点在花内连接到 j（用于花内边的代表选择）
vector<vector<int>> Leaf; // Leaf[f] : 花 f 的成员；若 f<=sizev, Leaf[f] 包含一个元素 f（约定）
vector<int> Match, Pre, Mark, Dad, Visit, Slack;
vector<ll> Expect;
queue<int> Q;
int GetLca(int x, int y) {
    static int tim = 0;
    if (++tim == INT_MAX) { tim = 1; fill(Visit.begin(), Visit.end(), 0); }
    while (x != 0) {
        Visit[x] = tim;
        x = Dad[Match[x]];
        if (x != 0) x = Dad[Pre[x]];
    }
    while (y != 0) {
        if (Visit[y] == tim) return y;
        y = Dad[Match[y]];
        if (y != 0) y = Dad[Pre[y]];
    }
    return 0;
}
inline ll CalcEdge(const Edges &e) { // 边代价 / slack 的计算（依据 Expect 顶标）
    return Expect[e.from] + Expect[e.to] - (ll)Eij[e.from][e.to].data * 2;
}
void UpdateSlack(int id, int now) { // 更新 now 对应的 Slack（记录能使 now 匹配的代表点 id）
    if (!Slack[now] || CalcEdge(Eij[id][now]) < CalcEdge(Eij[Slack[now]][now])) Slack[now] = id;
}
void CalcSlack(int now) {
    Slack[now] = 0;
    for (int i = 1; i <= sizev; ++i)
        if (Eij[i][now].data > 0 && Dad[i] != now && Mark[Dad[i]] == 0)
            UpdateSlack(i, now);
}
void JoinNodeToQueue(int now) {
    if (now <= sizev) { Q.push(now); return; }
    for (int v : Leaf[now]) JoinNodeToQueue(v);
}
void SetDadRec(int now, int dad) {
    Dad[now] = dad;
    if (now <= sizev) return;
    for (int v : Leaf[now]) SetDadRec(v, dad);
}
void SetMatch(int from, int to) { // 为 from 设置匹配到 to（处理花的展开/交错路径切换）
    Match[from] = Eij[from][to].to; // 保证 Eij[from][to].to 存储花/点对应“匹配的边”
    if (from <= sizev) return;
    int root = Root[from][Eij[from][to].from];
}

```

```

    auto it = find(Leaf[from].begin(), Leaf[from].end(), root);
    int place = (int)(it - Leaf[from].begin());
    if (place % 2 == 1) { reverse(Leaf[from].begin() + 1, Leaf[from].end()); place = (int)Leaf[from].size();
    for (int i = 0; i < place; ++i) SetMatch(Leaf[from][i], Leaf[from][i ^ 1]);
    SetMatch(root, to);
    rotate(Leaf[from].begin(), Leaf[from].begin() + place, Leaf[from].end());
}

void BlossomBreak(int now) { // 当一个花“无欲望”时，把其中不应保留的子花炸开
    for (int v : Leaf[now]) {
        if (v > sizev && !Expect[v]) BlossomBreak(v);
        else SetDadRec(v, v);
    }
    Dad[now] = 0;
}

void BlossomBuild(int from, int lca, int to) {
    int now = sizev + 1;
    while (now <= tot && Dad[now]) ++now;
    if (now > tot) ++tot;
    Expect[now] = Mark[now] = 0;
    Match[now] = Match[lca];
    Leaf[now].clear();
    Leaf[now].push_back(lca);
    for (int k = from; k != lca; k = Dad[Pre[ Dad[ Match[k] ] ] ]) {
        Leaf[now].push_back(k); Leaf[now].push_back(Dad[Match[k]]);
        JoinNodeToQueue(Dad[Match[k]]);
    }
    reverse(Leaf[now].begin() + 1, Leaf[now].end());
    for (int k = to; k != lca; k = Dad[Pre[ Dad[ Match[k] ] ] ]) {
        Leaf[now].push_back(k); Leaf[now].push_back(Dad[Match[k]]);
        JoinNodeToQueue(Dad[Match[k]]);
    }
    SetDadRec(now, now);
    for (int i = 1; i <= tot; ++i) { Eij[now][i].data = Eij[i][now].data = 0; Root[now][i] = 0; }
    for (int vex : Leaf[now]) {
        for (int j = 1; j <= tot; ++j)
            if (!Eij[now][j].data || CalcEdge(Eij[vex][j]) < CalcEdge(Eij[now][j]))
                Eij[now][j] = Eij[vex][j], Eij[j][now] = Eij[j][vex];
        for (int j = 1; j <= tot; ++j)
            if (Root[vex][j]) Root[now][j] = vex;
    }
    CalcSlack(now);
}

void GroupAugment(int x, int y) {
    while (true) {
        int z = Dad[Match[x]];
        SetMatch(x, y);
        if (z == 0) return;
        SetMatch(z, Dad[Pre[z]]);
        x = Dad[Pre[z]]; y = z;
    }
}

bool DealEdge(const Edges &e) {
    int from = Dad[e.from], to = Dad[e.to];

```

```

    if (Mark[to] == -1) {
        Pre[to] = e.from;
        Mark[to] = 1; Mark[ Dad[ Match[to] ] ] = 0;
        Slack[to] = Slack[ Dad[ Match[to] ] ] = 0;
        JoinNodeToQueue(Dad[ Match[to] ]);
    } else if (!Mark[to]) {
        int lca = GetLca(from, to);
        if (!lca) {
            GroupAugment(from, to); GroupAugment(to, from);
            for (int i = sizev + 1; i <= tot; ++i)
                if (Dad[i] == i && Expect[i] == 0) BlossomBreak(i);
            return true;
        } else BlossomBuild(from, lca, to);
    }
    return false;
}

void BlossomDelete(int now) {
    for (int v : Leaf[now]) SetDadRec(v, v);
    int root = Root[now][ Eij[now][ Pre[now] ].from ];
    auto it = find(Leaf[now].begin(), Leaf[now].end(), root);
    int place = (int)(it - Leaf[now].begin());
    if (place % 2 == 1) { reverse(Leaf[now].begin() + 1, Leaf[now].end()); place = (int)Leaf[now].size() - 1; }
    for (int i = 0; i < place; i += 2) {
        int from = Leaf[now][i], to = Leaf[now][i + 1];
        Pre[from] = Eij[to][from].from;
        Mark[from] = 1; Mark[to] = 0;
        Slack[from] = 0; CalcSlack(to); JoinNodeToQueue(to);
    }
    Mark[root] = 1; Pre[root] = Pre[now];
    for (int i = place + 1; i < (int)Leaf[now].size(); ++i) {
        int from = Leaf[now][i];
        Mark[from] = -1; CalcSlack(from);
    }
    Dad[now] = 0;
}

bool WorkPhase() {
    while (!Q.empty()) Q.pop();
    for (int i = 1; i <= tot; ++i) { Slack[i] = 0; Mark[i] = -1; }
    for (int i = 1; i <= tot; ++i)
        if (Dad[i] == i && !Match[i]) { Slack[i] = Pre[i] = Mark[i] = 0; JoinNodeToQueue(i); }
    if (Q.empty()) return false;
    while (true) {
        while (!Q.empty()) {
            int from = Q.front(); Q.pop();
            for (int to = 1; to <= sizev; ++to) {
                if (Eij[from][to].data > 0 && Dad[from] != Dad[to]) {
                    if (!CalcEdge(Eij[from][to])) {
                        if (DealEdge(Eij[from][to])) return true;
                    } else if (Mark[Dad[to]] != 1) UpdateSlack(from, Dad[to]);
                }
            }
        }
    }
    ll delta = Inf;
}

```

```

    for (int i = 1; i <= sizev; ++i) if (!Mark[Dad[i]]) delta = min(delta, Expect[i]);
    for (int i = sizev + 1; i <= tot; ++i) if (Dad[i] == i && Mark[i] == 1) delta = min(delta, Expect[i]);
    for (int i = 1; i <= tot; ++i)
        if (Dad[i] == i && Slack[i])
            if (Mark[i] == -1) delta = min(delta, CalcEdge(Eij[Slack[i]][i]));
            else if (Mark[i] == 0) delta = min(delta, CalcEdge(Eij[Slack[i]][i]) / 2);
    for (int i = 1; i <= sizev; ++i)
        if (Mark[Dad[i]] == 0) Expect[i] -= delta;
        else if (Mark[Dad[i]] == 1) Expect[i] += delta;
    for (int i = sizev + 1; i <= tot; ++i) if (Dad[i] == i)
        if (Mark[i] == 0) Expect[i] += delta * 2;
        else if (Mark[i] == 1) Expect[i] -= delta * 2;
    for (int i = 1; i <= sizev; ++i) if (!Expect[i]) return false;
    for (int i = 1; i <= tot; ++i)
        if (Dad[i] == i && Slack[i] && Dad[Slack[i]] != i && CalcEdge(Eij[Slack[i]][i]) == 0)
            if (DealEdge(Eij[Slack[i]][i])) return true;
    for (int i = sizev + 1; i <= tot; ++i)
        if (Dad[i] == i && Mark[i] == 1 && !Expect[i]) BlossomDelete(i);
}
return false;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> sizev >> m;
    tot = sizev;
    Eij.assign(MaxV, vector<Edges>(MaxV));
    Root.assign(MaxV, vector<int>(MaxV, 0));
    Leaf.assign(MaxV, vector<int>());
    Match.assign(MaxV, 0); Pre.assign(MaxV, 0); Mark.assign(MaxV, -1); Dad.assign(MaxV, 0);
    Visit.assign(MaxV, 0); Slack.assign(MaxV, 0); Expect.assign(MaxV, 0);
    for (int i = 1; i <= sizev; ++i) for (int j = 1; j <= sizev; ++j) Eij[i][j] = Edges(i, j, 0);
    int maxi = 0;
    for (int i = 0; i < m; ++i) {
        int u, v, w; cin >> u >> v >> w;
        Eij[u][v].data = Eij[v][u].data = w;
        maxi = max(maxi, w);
    }
    for (int i = 1; i <= sizev; ++i) { Match[i] = 0; Dad[i] = i; Leaf[i].clear(); Leaf[i].push_back(i); }
    for (int i = 1; i <= sizev; ++i) for (int j = 1; j <= sizev; ++j) Root[i][j] = (i == j ? i : 0);
    for (int i = 1; i <= sizev; ++i) Expect[i] = maxi;
    while (WorkPhase());
    ll ans = 0;
    for (int i = 1; i <= sizev; ++i) if (Match[i] && Match[i] < i) ans += Eij[i][Match[i]].data;
    cout << ans << '\n';
    for (int i = 1; i <= sizev; ++i) cout << Match[i] << ' ';
    cout << '\n';
    return 0;
}

```


一般图最大独立集

```
#include<cstdio>
#include<cstring>
#include<algorithm>
using namespace std;
const int Maxn=410;
int None[Maxn][Maxn],All[Maxn][Maxn],Some[Maxn][Maxn],Deg[Maxn];
bool Map[Maxn][Maxn];int ans;
inline void FindGraph(int pos,int al,int so,int no)
{
    if(al+so<=ans)return ;
    if(so==0&&no==0){ans=al;return ;}
    int pi=0;
    if(so){
        pi=Some[pos][1];
        for(int i=1;i<=al;i++)All[pos+1][i]=All[pos][i];
    }
    for(register int i=1;i<=so;i++){
        int now=Some[pos][i];
        if(!Map[pi][now])continue;
        int nno=0,nso=0;
        for(register int j=1;j<=so;j++)
            if(Map[now][Some[pos][j]])Some[pos+1][++nso]=Some[pos][j];
        for(register int j=1;j<=no;j++)
            if(!Map[now][None[pos][j]])None[pos+1][++nno]=None[pos][j];
        All[pos+1][al+1]=now;
        FindGraph(pos+1,al+1,nso,nno);
        Some[pos][i]=0;None[pos][++no]=now;
    }
}
bool Cmp(int a,int b) {return Deg[a]>Deg[b];}
int n,m;
int main(){
    scanf("%d%d",&n,&m);
    for(register int i=1;i<=n;i++){
        Deg[i]=n-1;
        Some[0][i]=i;
        Map[i][i]=Map[0][i]=Map[i][0]=true;
    }
    for(register int i=1;i<=m;i++){
        int x,y;scanf("%d%d",&x,&y);
        Map[x][y]=Map[y][x]=true;
        Deg[x]--;Deg[y]--;
    }
    sort(Some[0]+1,Some[0]+n+1,Cmp);
    Find_Graph(0,0,n,0);
    printf("%d\n",ans);
    return 0;
}
```

全图割

```
#include<stdio>
#include<cstring>
#include<algorithm>
using namespace std;

const int MaxN=610;

int Dist[MaxN][MaxN];
int Weight[MaxN],Ord[MaxN];bool Vis[MaxN],Res[MaxN];
int s,t,n,m;

inline int GetMin(int x){
    memset(Vis,false,sizeof(Vis));
    memset(Weight,0,sizeof(Weight));
    Weight[0]=-1;
    for(int i=1;i<=n-x+1;i++){
        int maxi=0;
        for(int j=1;j<=n;j++){
            if(!Vis[j]&&!Res[j]&&Weight[j]>Weight[maxi])
                maxi=j;
            Vis[maxi]=true;Ord[i]=maxi;
            for(int j=1;j<=n;j++){
                if(!Vis[j]&&!Res[j])
                    Weight[j]+=Dist[maxi][j];
            }
        }
        s=Ord[n-x],t=Ord[n-x+1];
        return Weight[t];
    }

inline int Solve(){
    int res=0x3f3f3f3f;
    for(int i=1;i<n;i++){
        res=min(res,GetMin(i));
        Res[t]=true;
        for(int j=1;j<=n;j++){
            Dist[s][j]+=Dist[t][j];
            Dist[j][s]+=Dist[j][t];
        }
    }
    return res;
}

int main(){
    scanf("%d%d",&n,&m);
    for(int i=1;i<=m;i++){
        int x,y,c;
        scanf("%d%d%d",&x,&y,&c);
        Dist[x][y]+=c;
        Dist[y][x]+=c;
    }
    printf("%d\n",Solve());
}
```

```
    return 0;  
}
```

如果要构造方案，找到使 res 最小时，让那个最后的点所代表的边放在一个连通块中